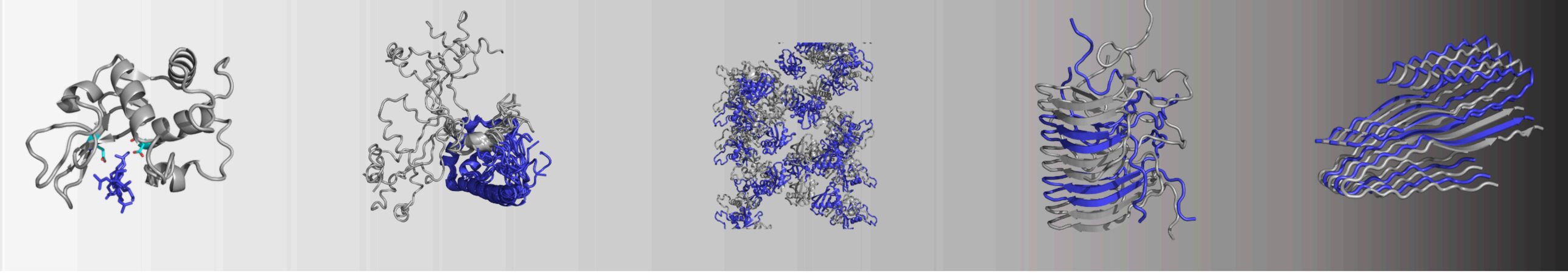
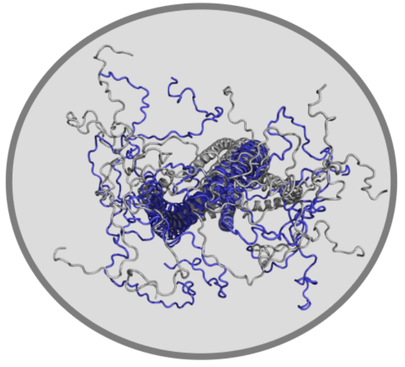


# Biological datasets for computational physics


Monika Fuxreiter

*Laboratory of Protein Interactions and Dynamics*
*Department of Biomedicine*
*University of Padova, Italy*

*http://biomed.unipd.it/people/fuxreiter-monika*
*http://protdyn.med.unideb.hu*

# Biological datasets for computational physics

## Biopython practical

**General applications and structural bioinformatics**

*http://biomed.unipd.it/people/fuxreiter-monika*
*http://protdyn.med.unideb.hu*

# What is biopython?


Python tools for Computational Molecular Biology

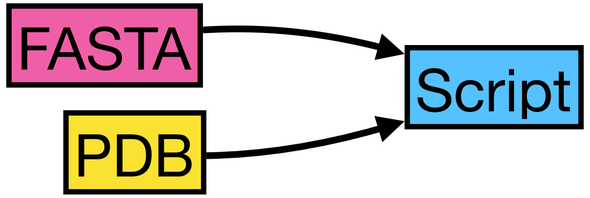
**Sequence analysis**

DNA coding strand (aka Crick strand, strand +1)

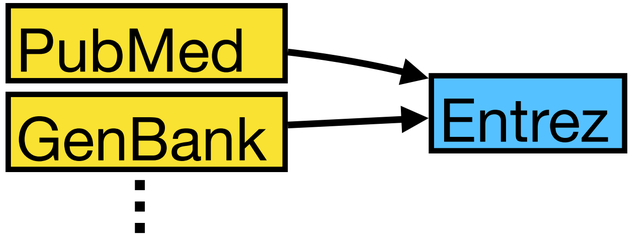
```
5'  ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG  3'
    ||||||||||||||||||||||||||||||||||||||||
3'  TACCGGTAACATTACCCGGCGACTTTCCCACGGGCTATC  5'
```

DNA template strand (aka Watson strand, strand −1)

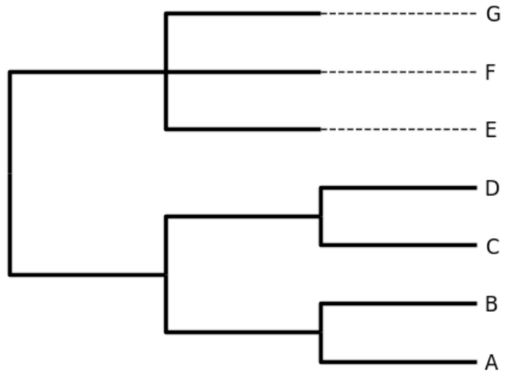
**File parsing**




**Database access**




**Phylogenetics**

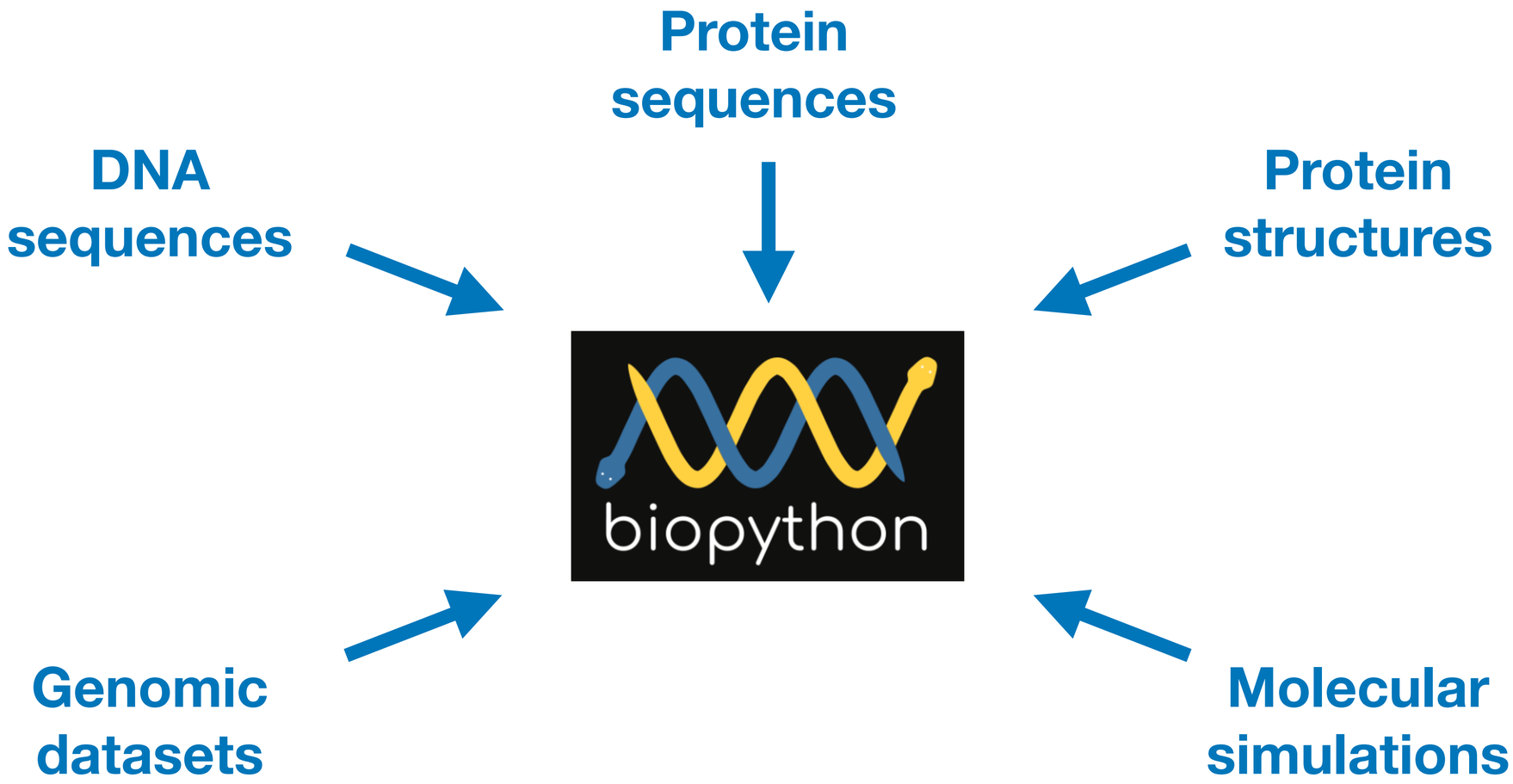



**Structural bioinformatics**

**Alignment**

```
1 MGDVEKGKKIFIMKCSQCHTVEKGGKHKTGPNLHGLFGRKTGQAPGYSYT      50
  ||||||||||:.||:|||||||||||||||||||||||.|:|||
1 MGDVEKGKKIFVQKCAQCHTVEKGGKHKTGPNLHGLFGRKTGQAAGFSYT      50
```

# Why biopython?




Wide variety of data sources, all with different formats and methods

# Why biopython?

**Biological Datasets**

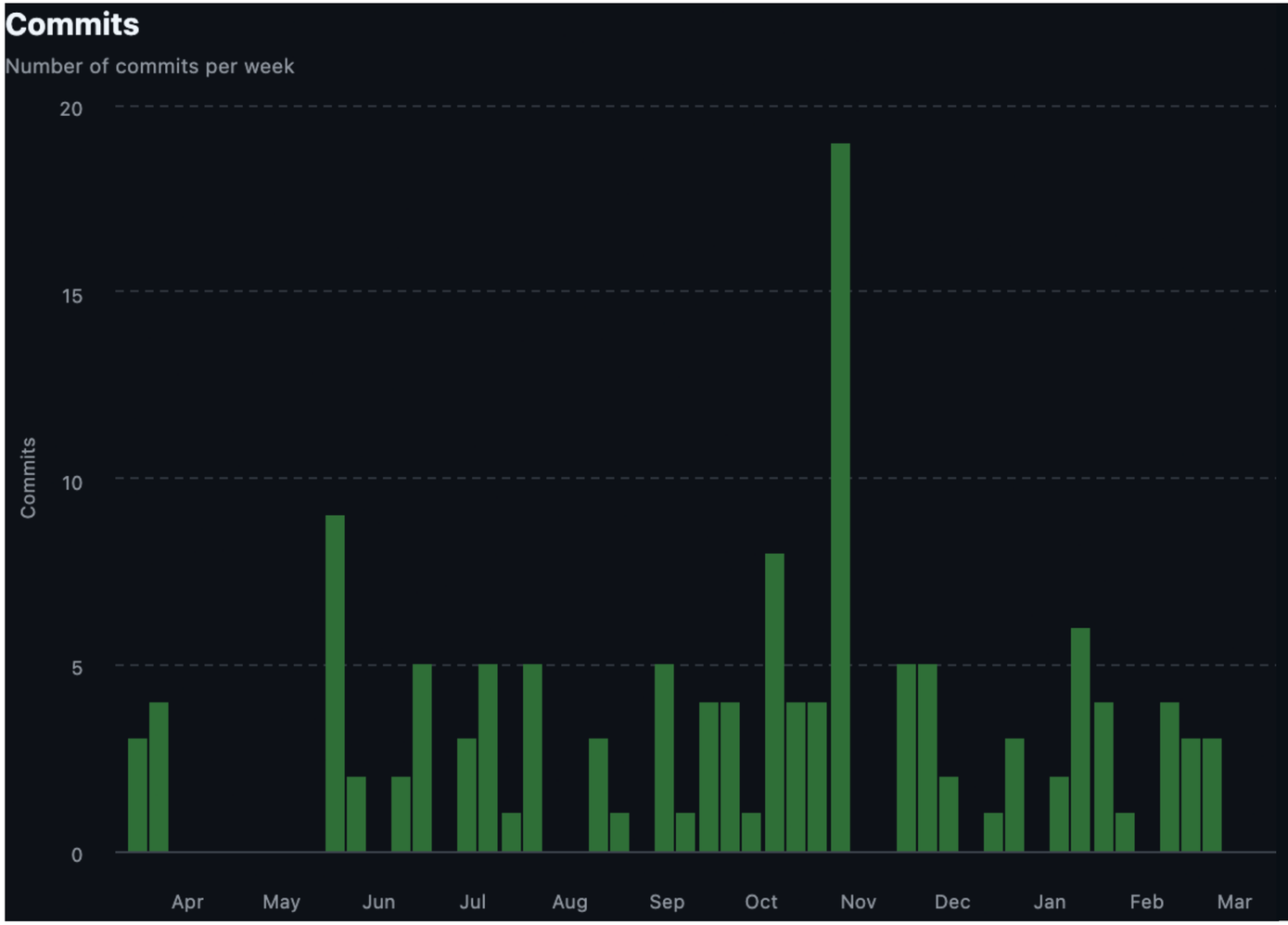

**Integrated workflow**

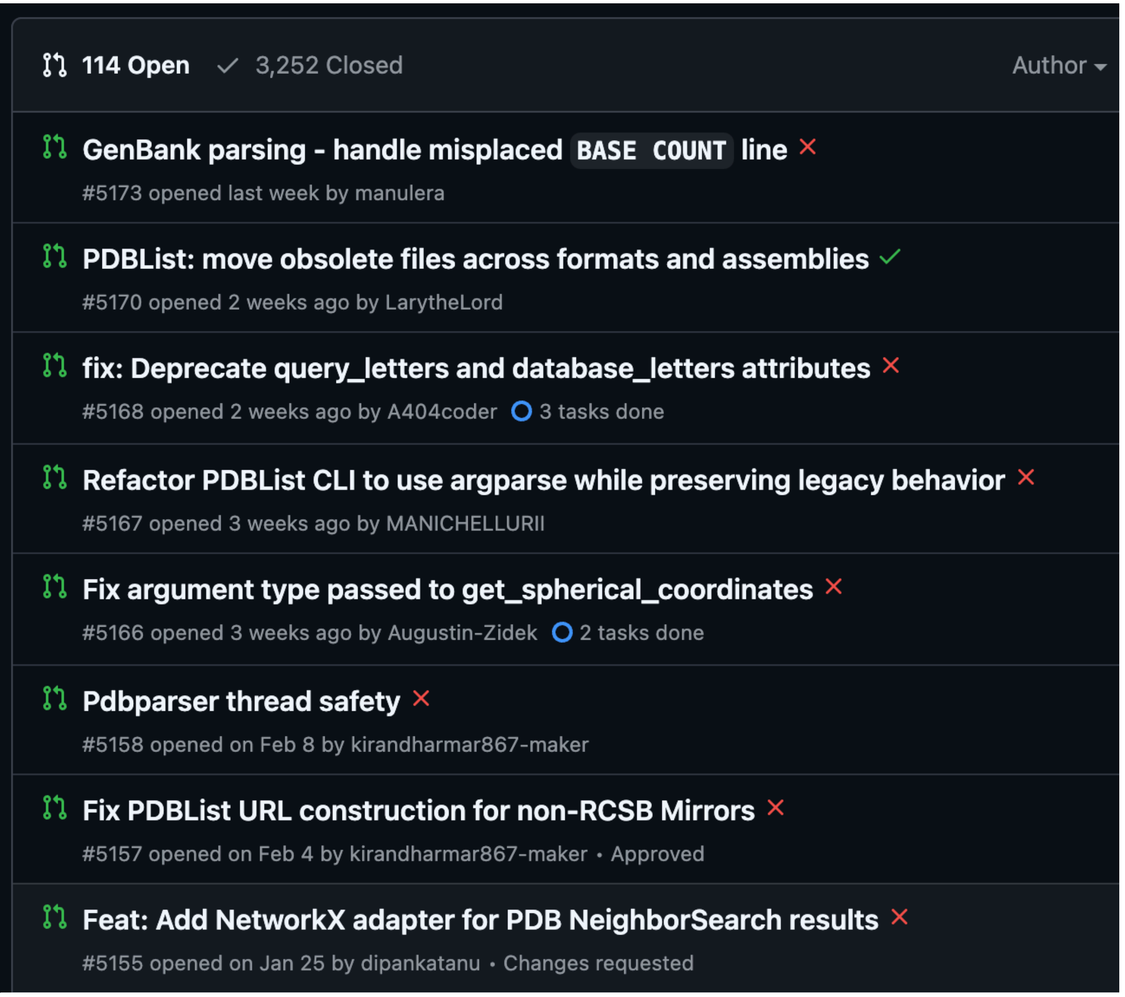
**ML techniques**

Biopython can act as a bridge between our biological data and the latest tools and algorithms

# Why biopython?

Still in active development, with requests for improvements / bug fixes…



# Biological sequences

```
# Load modules
from Bio.Seq import Seq
```

```
# Let us define a dummy DNA sequence
dna = Seq("ATGCGTACG")
print(f'Original sequence:         {dna}')
# We can obtain its reverse complement (DNA and RNA)
print(f'Reverse complement:        {dna.reverse_complement()}')
print(f'Reverse complement (RNA): {dna.reverse_complement_rna()}')
# As well as its translation
print(f'\nTranslated sequence: {dna.translate()}')
```

```
Original sequence:         ATGCGTACG
Reverse complement:        CGTACGCAT
Reverse complement (RNA): CGUACGCAU

Translated sequence: MRT
```

```
# Let us define a dummy protein sequence
prot = Seq("HHARARMLLIVAAR")

# We can easily count a given amino acid
print('Number of A:', prot.count('A'))

# And a given pattern
print('Number of AR:', prot.count('AR'))
```

```
Number of A: 4
Number of AR: 3
```

# Biological data I/O

```
# Load modules
from Bio import SeqIO
```

```
# Read information from a FASTA file
path = './Data/ACE2_YEAST.fasta'
for record in SeqIO.parse(path, "fasta"):
    print(record.id)
    print(record.seq)
```

```
sp|P21192|ACE2_YEAST
MDNVVDPWYINPSGFAKDTQDEEYVQHHDNVNPTIPPPDNYILNNENDDGLDNLLGMDYYNIDDLLTQELRDLDIPLVPSPKTGDGSSDKKNIDRTWNLGDENNKVSHYSKKSMSSHKRGLSGTAIFGFLGHNKTLSISSL
QQSILNMSKDPQPMELINELGNHNTVKNNNDDFDHIRENDGENSYLSQVLLKQQEELRIALEKQKEVNEKLEKQLRDNQIQQEKLRKVLEEQEEVAQKLVSGATNSNSKPGSPVILKTPAMQNGRMKDNAIIVTTNSANGG
YQFPPPTLISPRMSNTSINGSPSRKYHRQRYPNKSPESNGLNLFSSNSGYLRDSELLSFSPQNYNLNLDGLTYNDHNNTSDKNNNDKKNSTGDNIFRLFEKTSPGGLSISPRINGNSLRSPFLVGTDKSRDDRYAAGTFTP
RTQLSPIHKKRESVVSTVSTISQLQDDTEPIHMRNTQNPTLRNANALASSSVLPPIPGSSNNTPIKNSLPQKHVFQHTPVKAPPKNGSNLAPLLNAPDLTDHQLEIKTPIRNNSHCEVESYPQVPPVTHDIHKSPTLHSTS
PLPDEIIPRTTPMKITKKPTTLPPGTIDQYVKELPDKLFECLYPNCNKVFKRRYNIRSHIQTHLQDRPYSCDFPGCTKAFVRNHDLIRHKISHNAKKYICPCGKRFNREDALMVHRSRMICTGGKKLEHSINKKLTSPKKS
LLDSPHDTSPVKETIARDKDGSVLMKMEEQLRDDMRKHGLLDPPPSTAAHEQNSNRTLSNETDAL
```

We can parse large datasets without fully loading them into memory.

# Biological data I/O

Lots of different file formats can be parsed…

```
# Read multiple records in plain text Swiss-Prot aka UniProt format.
path = './Data/uniprotkb_yeast_transcription_factor_AN_2026_03_10.txt'
records = SeqIO.to_dict(SeqIO.parse(path, 'swiss'))
print(f'Identified {len(records)} entries\n')

# We can now access the data by uniprot id
unip_id = 'P11747'
print('Example entry:')
print('UniProt ID:', unip_id)
print('Name:', records[unip_id].name)
print('Sequence:', records[unip_id].seq)
```

```
Identified 82 entries

Example entry:
UniProt ID: P11747
Name: TAF13_YEAST
Sequence: MSRKLKKTNLFNKDVSSLLYAYGDVPQPLQATVQCLDELVSGYLVDVCTNAFHTAQNSQRNKLRLEDFKFALRKDPIKLGRAEELIATNKLITEAKKQFNETDNQNSLKRYREEDEEGDEMEEDEDEQQVT
DDDEEAAGRNSAKQSTDSKATKIRKQGPKNLKKTKK
```

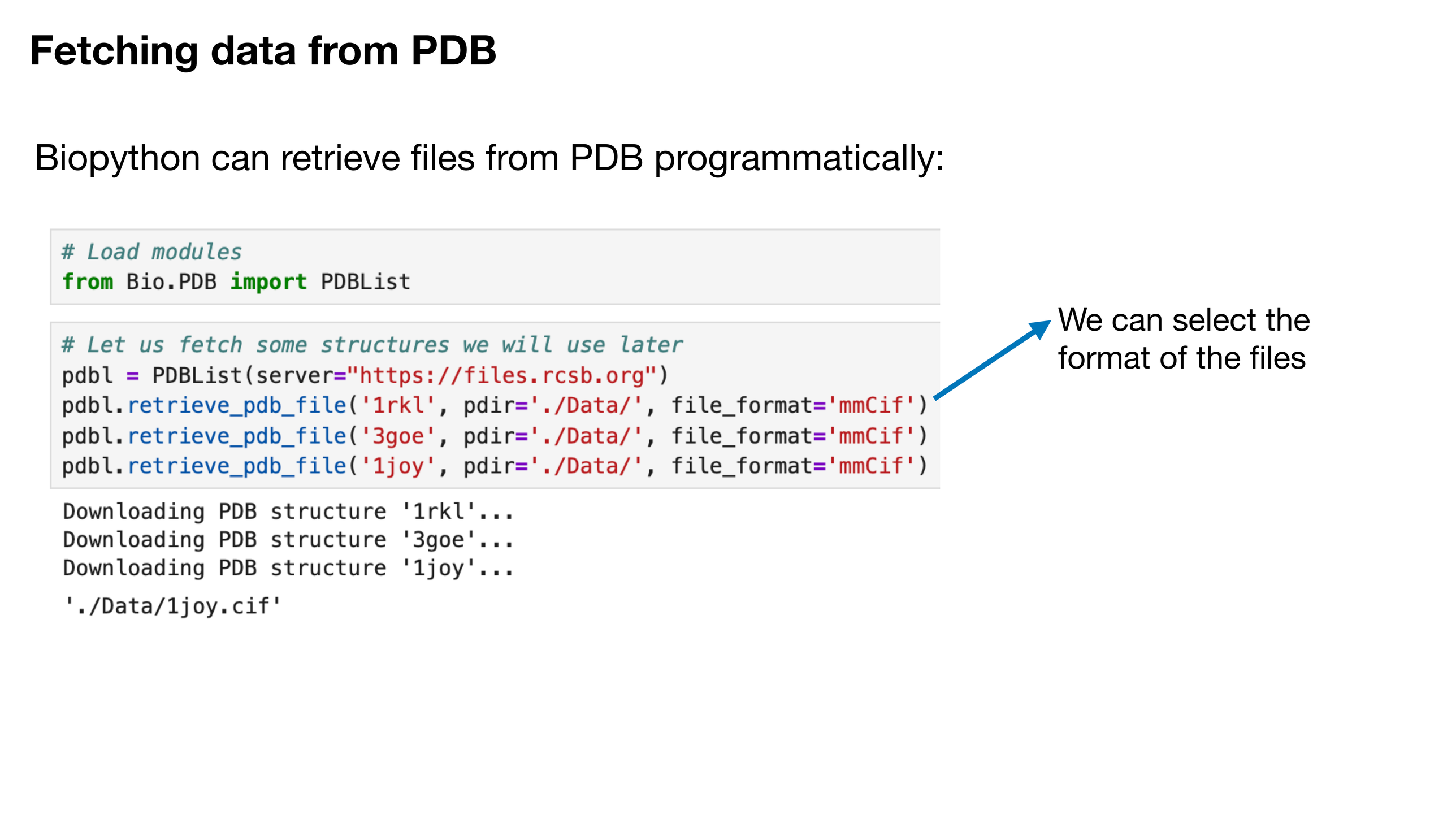
# Fetching data from PDB

Biopython can retrieve files from PDB programmatically:

```
# Load modules
from Bio.PDB import PDBList
```

```
# Let us fetch some structures we will use later
pdbl = PDBList(server="https://files.rcsb.org")
pdbl.retrieve_pdb_file('1rkl', pdir='./Data/', file_format='mmCif')
pdbl.retrieve_pdb_file('3goe', pdir='./Data/', file_format='mmCif')
pdbl.retrieve_pdb_file('1joy', pdir='./Data/', file_format='mmCif')
```

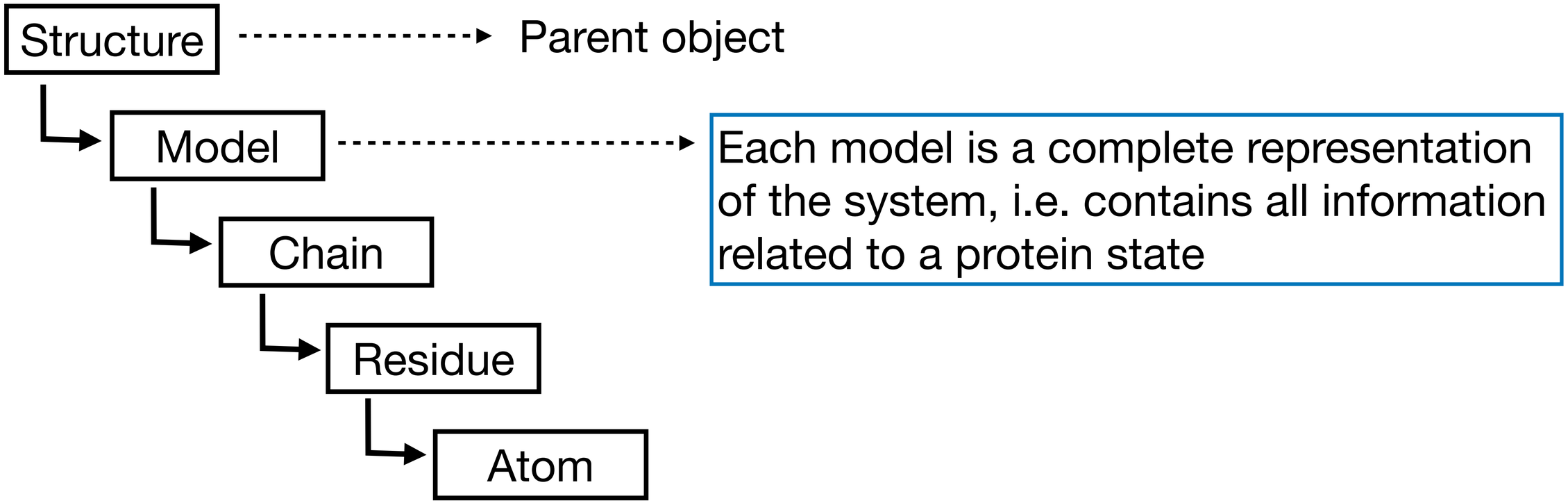
```
Downloading PDB structure '1rkl'...
Downloading PDB structure '3goe'...
Downloading PDB structure '1joy'...
'./Data/1joy.cif'
```



# Reading protein structures

Biopython represents structures hierarchically:

```
# Load modules
from Bio.PDB import PDBParser, MMCIFParser
```



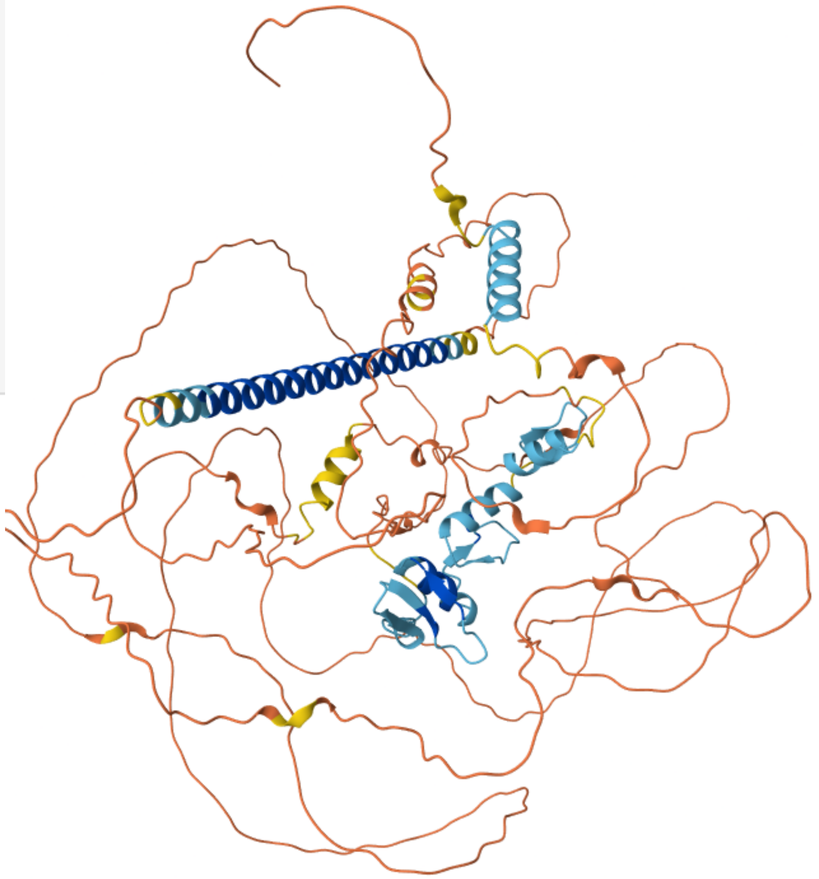
# Reading protein structures

We can load PDB files (also from AlphaFold):

```
# Initialize parser
parser = PDBParser()
# Read our PDB structure
structure = parser.get_structure("ACE2_yeast", "./Data/AF-P21192-F1-model_v6.pdb")

# Let's investigate the structure of the object
print('Structure (ACE2_YEAST):')
print('-> Models:', list(structure))
print('   -> Chains:', list(structure[0]))
print('      -> Residues:', len(structure[0]['A']))
print('         -> Atoms:', len(list(structure[0]['A'].get_atoms())))
```

```
Structure (ACE2_YEAST):
-> Models: [<Model id=0>]
   -> Chains: [<Chain id=A>]
      -> Residues: 770
         -> Atoms: 6088
```

ACE2_YEAST (P21192)



Note: chains are fetched with a key instead of an index

https://alphafold.ebi.ac.uk/entry/P21192

# Reading protein structures

We can also load mmCIF files:

```
# Initialize parser
parser = MMCIFParser()
# Read our mmCIF structure
structure = parser.get_structure("1rkl", "./Data/1RKL.cif")

# Let's investigate the structure of the object
print('Structure (NMR structure of yeast oligosaccharyltransferase subunit Ost4p):')
print('-> Models:', len(list(structure)))
print('   -> Chains:', len(list(structure[0])))
print('      -> Residues:', len(structure[0]['A']))
print('         -> Atoms:', len(list(structure[0]['A'].get_atoms())))
```

```
Structure (NMR structure of yeast oligosaccharyltransferase subunit Ost4p):
-> Models: 20
   -> Chains: 1
      -> Residues: 36
         -> Atoms: 525
```

Yeast oligosaccharyl-transferase subunit Ost4p

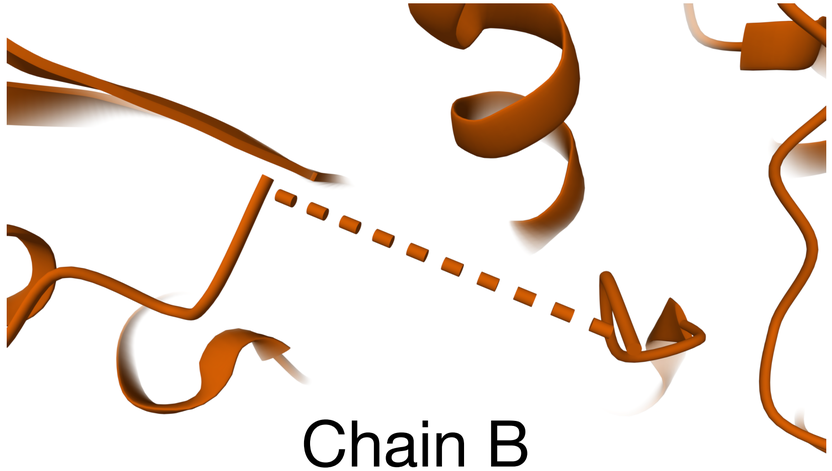
https://www.rcsb.org/structure/1RKL

# Chain discontinuities

If there are missing residues in the experimental data, biopython issues a warning when attempting to build the chains:

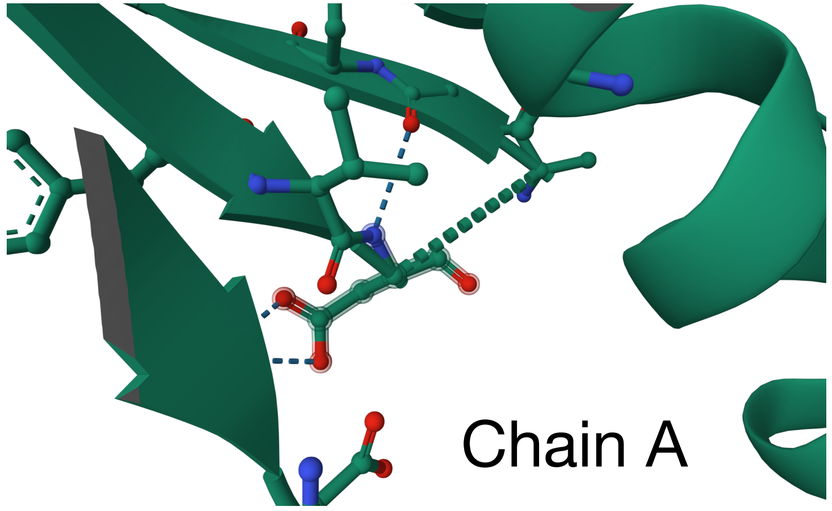
```
# Initialize parser
parser = MMCIFParser()
# Read our mmCIF structure with missing residues
structure = parser.get_structure("1fk9", "./Data/1fk9.cif")

# If we try to iterate over the residues we will get a warning
residues = list(structure[0].get_residues())
```

```
/Users/francoaquistapace/biopython-test/env/lib/python3.8/site-packages/Bio/PDB/StructureBuilder.py:89: PDBConstructionWarning: WARNING: Chai
n A is discontinuous at line 7606.
  warnings.warn(
/Users/francoaquistapace/biopython-test/env/lib/python3.8/site-packages/Bio/PDB/StructureBuilder.py:89: PDBConstructionWarning: WARNING: Chai
n B is discontinuous at line 7690.
  warnings.warn(
```



Chain B

PDB:1FK9



Chain A

# Separating protein residues from other molecules

We can separate the protein from the ions / ligands / water:

```
# How can we understand if a residue is representing a heteroatom?
all_ids = []
for i, r in enumerate(residues):
    all_ids.append(r.get_id()[0])

print('Unique residue IDs in 1fk9:')
print(np.unique(all_ids))
```

```
Unique residue IDs in 1fk9:
[' ' 'H_CSD' 'H_EFZ' 'W']
```

We have H_CSD for a PTM residue, H_EFZ for the EFAVIRENZ ligand and W for water.

```
# Now we can extract protein residues
prot_residues = []
for r in residues:
    # Skip H_EFZ and W residues
    if r.get_id()[0] in ['H_EFZ', 'W']:
        continue

    prot_residues.append(r)
```

Similarly, we could use the same approach to isolate the residue corresponding to EFZ.

# Adding custom properties to a model

We can get / set many properties of a structure and its children objects:

```python
# Assume we have some computed values we want to assign to
# our atoms in the B-factor attribute
custom_bfactor = np.random.uniform(size=len(list(structure[0].get_atoms())))

# Assign the new values with set_bfactor(), making sure we
# iterate over all models for consistency
for mdl in structure:
    for i, a in enumerate(structure[0].get_atoms()):
        a.set_bfactor(custom_bfactor[i])

# Print new values for the same atoms as before
print('\nCustom B-factor values:')
for i, a in enumerate(structure[0].get_atoms()):
    print(i, a.get_bfactor())
    if i >= 4:
        break

# We can write the structure with our custom B-factor to
# a new file
io = MMCIFIO()
io.set_structure(structure)
io.save('./Data/1rkl_custom.cif')
```

```
Original B-factor values:
0 0.0
1 0.0
2 0.0
3 0.0
4 0.0

Custom B-factor values:
0 0.6042194815302442
1 0.9417028697026933
2 0.7406086728490405
3 0.23459446958142616
4 0.2709571469484109
```

# Building the sequence from the structure

We can access residue names:

```
# Let us load the 3GOE PDB entry
parser = MMCIFParser()
structure = parser.get_structure("3goe", "./Data/3GOE.cif")

# What residues do we have in the structure?
all_residues = [r.get_resname() for r in structure[0]['A'].get_residues()]
print(np.unique(all_residues))
```

```
['ARG' 'ASN' 'ASP' 'CA' 'CYS' 'GLN' 'GLU' 'GLY' 'HIS' 'HOH' 'ILE' 'LEU'
 'LYS' 'PHE' 'PRO' 'SER' 'THR' 'TRP' 'TYR' 'VAL']
```

Notice that we have ions (CA) and water (HOH) among our residues…

# Building the sequence from the structure

If we build our sequence naively, we include the ions and water:

```
# Build sequence naively
seq = ''.join([d3to1[r] if r in d3to1 else 'X' for r in all_residues])
print('Sequence (including ions and water):\n', seq)
```

```
Sequence (including ions and water):
 HHHHHHKLITLLLRSSKSEDLRLSIPVDFTVKDLIKRYCTEVKISFHERIRLEFEGEWLDPNDQVQSTELEDEDQVSVVLXXX...
XXXXXXXXXXXXXXXXXXXXX
```

Instead of implementing our own logic, we can use the polypeptide module:

```
# Load modules
from Bio.PDB.Polypeptide import PPBuilder
```

```
# Build sequence with biopython
ppb = PPBuilder()
for pp in ppb.build_peptides(structure):
    print(pp.get_sequence())
```

```
HHHHHHKLITLLLRSSKSEDLRLSIPVDFTVKDLIKRYCTEVKISFHERIRLEFEGEWLDPNDQVQSTELEDEDQVSVVL
```

PDB: 3GOE

# Sequence analysis tools

Biopython offers pre-defined tools to help us analyze the sequence:

```
# Load modules
from Bio.SeqUtils.ProtParam import ProteinAnalysis
```

```
# Get our sequence
ppb = PPBuilder()
seq = ppb.build_peptides(structure)[0].get_sequence()
seq_analysis = ProteinAnalysis(seq)

# Let us print the amino acid percentages
print('Amino acid percentages for entry 3GOE:')
print(seq_analysis.get_amino_acids_percent())

# We could also, for instance, calculate the aromaticity
# of the sequence, according to Lobry, 1994: P(F + W + Y)
print(f'\nAromaticity for entry 3GOE: {seq_analysis.aromaticity():.4f}')

# Additionally, we can get the grand average of hydropathy,
# as defined by Kyte and Doolittle (1982)
print(f'\nGRAVY for entry 3GOE: {seq_analysis.gravy():.2f} (strongly hydrophilic)')
```

```
Aromaticity for entry 3GOE: 0.0625

GRAVY for entry 3GOE: -0.55 (strongly hydrophilic)
```

Amino acid percentages

| Res | % |
|---|---|
| A | 0.0000 |
| C | 0.0125 |
| D | 0.0875 |
| E | 0.1125 |
| F | 0.0375 |
| G | 0.0125 |
| H | 0.0875 |
| I | 0.0625 |
| K | 0.0625 |
| L | 0.1375 |
| M | 0.0000 |
| N | 0.0125 |
| P | 0.0250 |
| Q | 0.0375 |
| R | 0.0625 |
| S | 0.0875 |
| T | 0.0500 |
| V | 0.0875 |
| W | 0.0125 |
| Y | 0.0125 |

(Alphabetical, not the best format…)

# Sequence analysis tools

We should always seek to understand the analysis we are running:

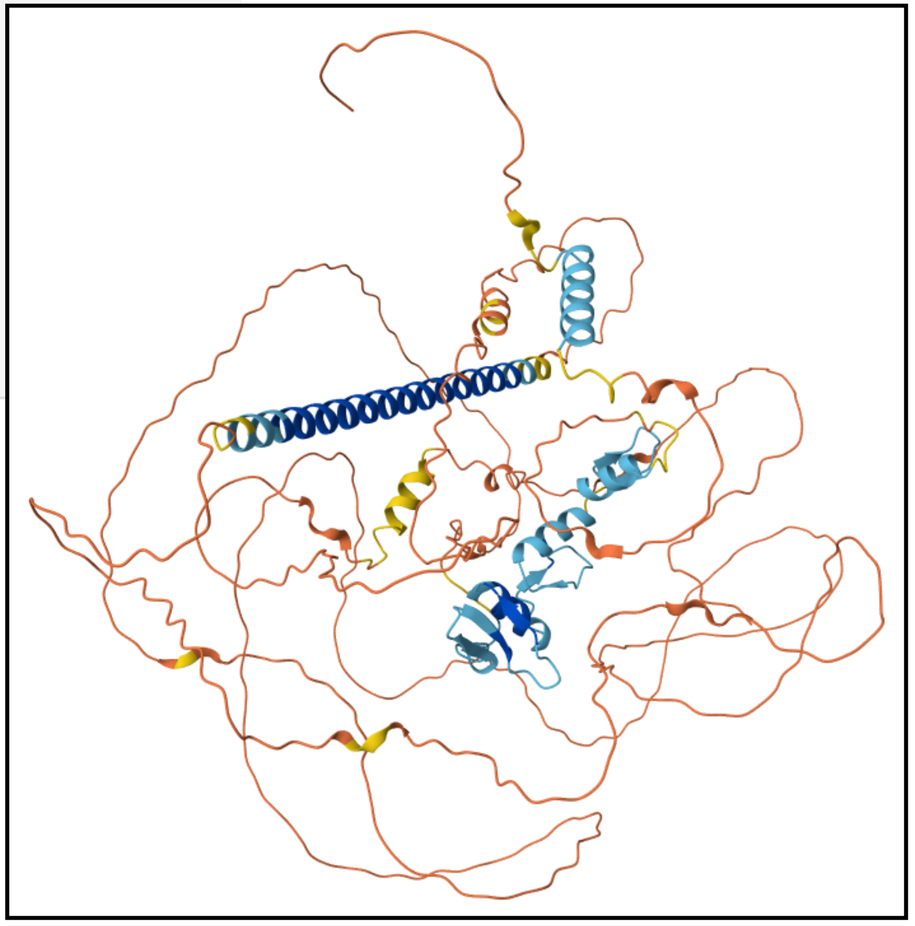
```
parser = PDBParser()
structure = parser.get_structure("ACE2_yeast", "./Data/AF-P21192-F1-model_v6.pdb")

ppb = PPBuilder()
seq = ppb.build_peptides(structure)[0].get_sequence()
seq_analysis = ProteinAnalysis(seq)

print('Secondary structure fraction of ACE2_YEAST:')
sec_struct_fraction = seq_analysis.secondary_structure_fraction()
for i, s in enumerate(['Helix', 'Turn', 'Sheet']):
    print(f'{s}: {sec_struct_fraction[i]:.2f} %')
```

```
Secondary structure fraction of ACE2_YEAST:
Helix: 0.27 %
Turn: 0.39 %
Sheet: 0.30 %
```

From a grouping of amino acids

However, predicted to be mostly disordered

ACE2_YEAST (P21192)



# Calculating residue distances

We can access the coordinates for each atom in each residue. We will use the positions of the C$\alpha$ to measure residue distances:

```
# We can access the carbon alpha for each residue
ca1 = structure[0]["A"][1]["CA"]
ca2 = structure[0]["A"][80]["CA"]

# Biopython implements distance calculation internally
dist = ca1 - ca2
print('Distance (biopython):', dist)

# We can check that it works properly using numpy
dist_np = np.linalg.norm(ca1.get_coord() - ca2.get_coord())
print('Distance (numpy):    ', dist_np)
```

```
Distance (biopython): 37.387215
Distance (numpy):     37.387215
```


**Exercise:**

Extract the carbon alpha coordinates of the protein without the N-terminal histidine tag (first 6 residues) nor the water and ion atoms (hetero atoms).

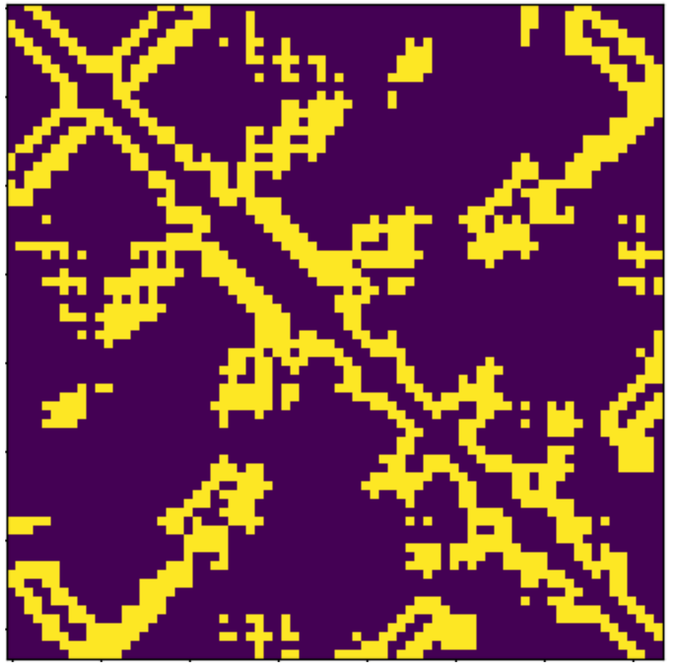
# Calculating residue distances

**Protein contact map:**

- Represents the distance between all possible amino acid residue pairs of a protein structure, using a binary matrix:

$$M_{ij} = \begin{cases} 1 & \text{if } d_{ij} \in (6\text{Å}, 12\text{Å}) \\ 0 & \text{if } d_{ij} \notin (6\text{Å}, 12\text{Å}) \end{cases}$$

- Here we represent the positions of the residues through the position of the C$\alpha$.



**Exercise:**

Create a all-to-all contact map for the protein residues using the coordinates array we generated previously. Use 6 Å and 12 Å as the inner and outer cutoffs, respectively.

# Superimposing multiple models

We can align multiple models to a chosen reference:

```
# Let us align the last 20 models to the first one:
# Initialize superimposer
superimposer = Superimposer()

# Set reference model
ref_model = structure[0]
print(f'RMS values using model {ref_model.id} as reference:')
for model in structure:
    # Skip model if it is the same as the ref.
    if model.id == ref_model.id:
        continue

    # Get CA atoms
    ref_atoms = [res['CA'] for res in ref_model.get_residues()]
    alt_atoms = [res['CA'] for res in model.get_residues()]

    # Align the alternative atoms
    superimposer.set_atoms(ref_atoms, alt_atoms)
    superimposer.apply(model.get_atoms())

    print(f'Model {model.id}: {superimposer.rms:.2f}')
```

```
RMS values using model 0 as reference:
Model 1: 2.03
Model 2: 2.86
```

⋮

PDB: 1JOY

# Superimposing multiple models

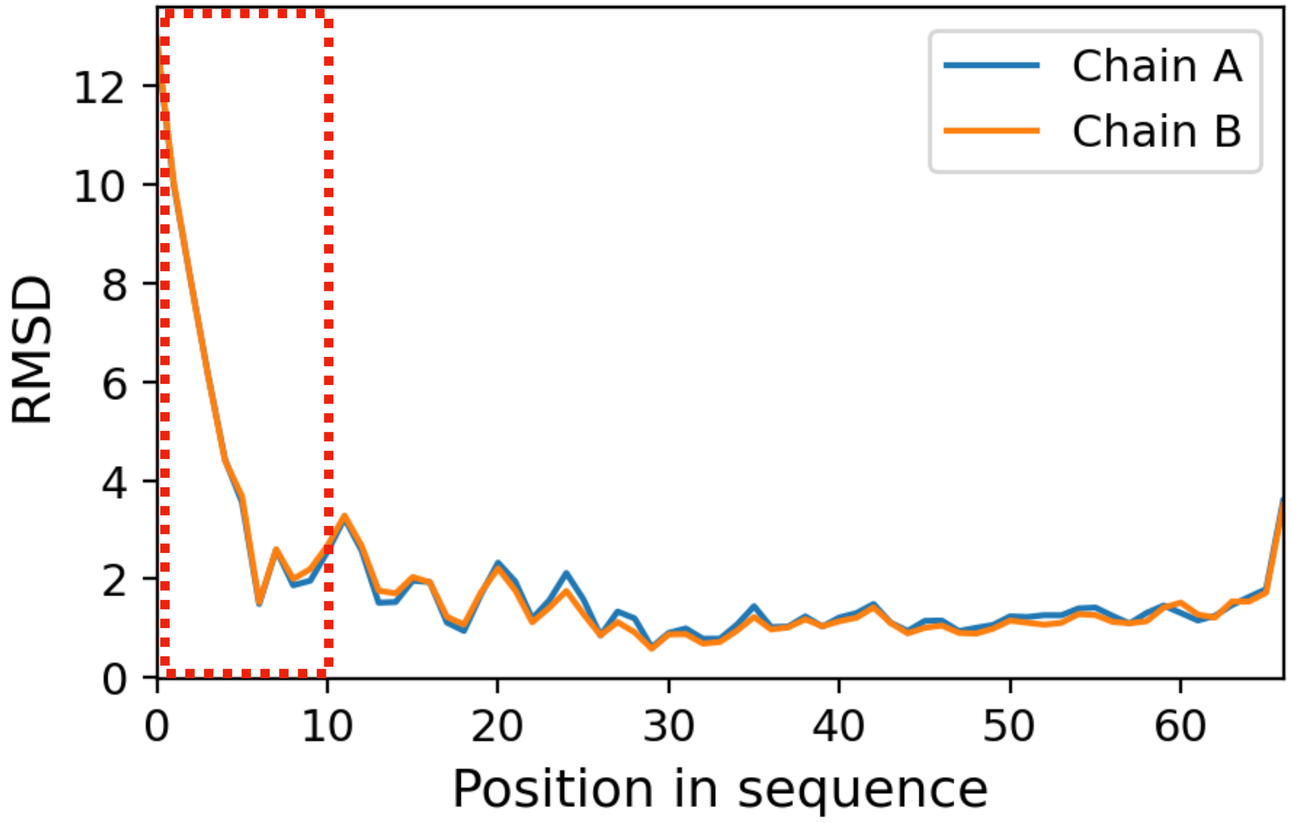
With the models aligned, we can estimate the RMSD for each residue:

```
# Now, we can estimate the RMSD for each residue
res_rmsd_A = dict()
res_rmsd_B = dict()
for chain in ref_model:
    for res in chain.get_residues():
        # Initialize deviation values for each residue
        res_deviations = []

        for model in structure:
            # Skip reference model
            if model.id == ref_model.id:
                continue

            # Get alt. chain and residue
            alt_chain = [ch for ch in model if ch.id == chain.id][0]
            alt_res = [r for r in alt_chain.get_residues() if r.id == res.id][0]

            # Get deviation (here we only consider CA positions)
            res_deviations.append(alt_res['CA'] - res['CA'])

        # Calculate residue RMSD and save for each chain
        if chain.id == 'A':
            res_rmsd_A[res.id] = np.sqrt(np.mean(np.square(res_deviations)))
        else:
            res_rmsd_B[res.id] = np.sqrt(np.mean(np.square(res_deviations)))
```

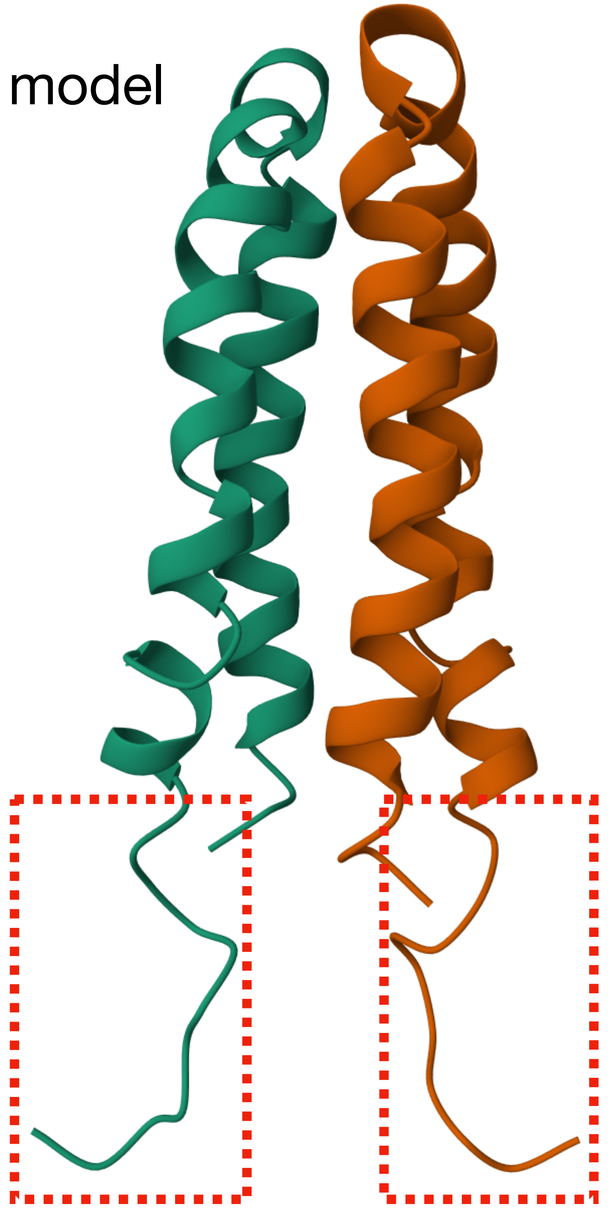
Be careful to ensure that we are not comparing residues from different chains

# Superimposing multiple models

With the models aligned, we can estimate the RMSD for each residue:




Ref. model



# Calculating geometric information

**Radius of gyration:** it can be used to describe the geometric conformation of a polymer chain. A smaller $R_G^2$ indicates a globular conformation, while a larger value indicates an elongated geometry:

$$R_G^2 = \frac{1}{N}\sum_{k=1}^{N}\left|\vec{r}_k - \vec{r}_{CM}\right|^2$$

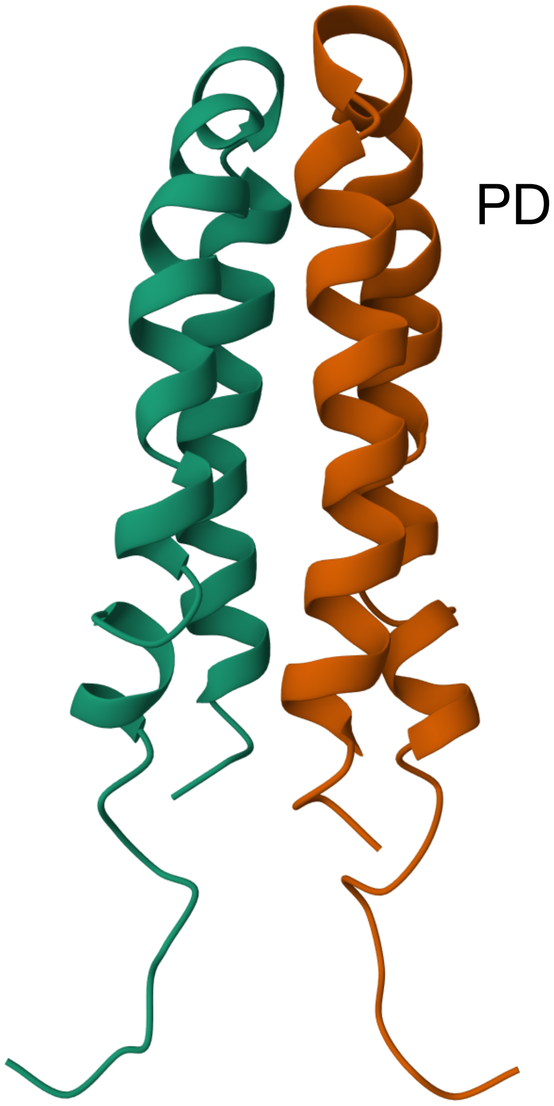
$\vec{r}_{CM}$: geometric center of mass

```python
def get_radius_of_gyration(mdl):
    # Get residues, excluding water, ligands and ions
    res = [r for r in mdl.get_residues() if r.get_id()[0] == ' ']

    # Extract coordinates from the CA of each residue
    X = np.array([r['CA'].get_coord() for r in res])

    # Get the geometric center of mass
    r_cm = np.mean(X, axis=0)

    # Calculate the radius of gyration and return
    R2_G = np.mean(np.square(np.linalg.norm(X - r_cm, axis=0)))
    return R2_G
```

Not pre-defined, but easy to implement thanks to biopython's data structures

# Calculating geometric information

PDB: 3GOE



PDB: 1JOY

(Globular) $R_G^2 = 3944\ \text{Å}^2$ $<$ $R_G^2 = 14595\ \text{Å}^2$ (Extended)

# Hands-on practice

Let us analyze and describe the structure of **human hemoglobin**:

1. Download the mmCIF file from PDB and parse it
2. Extract the sequence of the protein subunits and obtain the amino acid percentages
3. Identify, for each chain, the residue that is closest to a Fe atom
   - Remember to account for all the atoms in each residue, not only the C$\alpha$
4. Calculate the radius of gyration of the individual chains, as well as for the complete structure
5. (Optional) Compare the structure with that of hemoglobin when bound to oxygens (PDB: 1GZX)

PDB: 6FQF